

Why Runes

A Ruby agent harness with a fabric, an identity, a memory and a face

Runes 0.3.0 | 11 September 2026 | github.com/runes-harness/runes

A Ruby agent harness with a fabric, an identity, a memory, and a face. Fun to build, and quietly one of the more consequential things you could be building right now.

TL;DR

- **Runes runs fleets of LLM agents** over a swappable message fabric (in-process <-> MQTT 3.1.1 <-> MQTT 5), with A2A for discovery and MCP for tools.
- **Every workflow verb is a plugin.** `cmd`, `ruby`, `chat`, `agent`, `map`, `repeat`, `call` - seven runes, between 68 and 693 lines each, all replaceable, and a new one takes about thirty lines.
- **It runs Shopify Roast workflows** unmodified.** The shipped example is Roast's README example, byte for byte, and the test suite reads that exact file.
- **It treats the boring parts as the product:** capability guard, signed envelopes, a durable journal, verify-and-resume, and a Rails observatory that watches the whole fleet.
- **14 265 lines of lib, 10 646 lines of tests, 567 tests, zero network in the suite.** Ruby 4.0.4, Rails 8.1.3.1, SQLite, mosquito on localhost.

60 seconds: what it actually is

A workflow is a plain Ruby file that is `instance_eval`'d - no DSL gem, no YAML, no graph builder:

```
execute do
  cmd(:recent_changes) { "git diff --name-only HEAD~5..HEAD" }

  agent(:review) do
    files = cmd!(:recent_changes).lines
    "Review these for security, performance and maintainability:\n#{files.join("\n")}"
  end

  chat(:summary) do
    "Summarize this for non-technical stakeholders:\n\n#{agent!(:review).response}"
  end
end
```

```
bin/runes-workflow execute examples/analyze_codebase.rb
```

`cmd!(:recent_changes).lines` inside the agent block is a method call, not a variable - inside a step, `self` is the step's input context. That single design decision is why the syntax stays readable as workflows grow, and it is Roast's, not ours.

Underneath, the same file drives a fleet over MQTT, where agents are discoverable, addressable, and auditable:

```
runes/agents/<id>/card | status | tasks/<req>/response
runes/prompts                               <- broadcast work
runes/prompts/<req>/progress -> response    <- one shared group, no claim race
runes/tools/<tool>/request | response | error
runes/ log/prompts                          <- durable journal
$a2a/v1/discovery/<org>/<unit>/<agent_id>
$a2a/v1/tasks/<org>/<unit>/<agent_id>
```

Ten things that make this genuinely fun to build

1. Deleting the clever part made it stronger. 0.3 removed the entire claim/lease consensus protocol - about 250 lines of "who gets this work?" bookkeeping - and replaced it with MQTT 5 shared subscriptions. The broker now does exactly-once distribution natively. We proved it live: 200 messages, two clients in one shared group, 100/100 each, no dupes, no losses. **The lesson is the fun part: the best distributed-systems code is the code you get to delete.**

2. A hand-rolled MQTT 5 client, and it's only 899 lines. The whole adapter - CONNECT/CONNACK property probing, PUBLISH properties, \$share subscriptions, keepalive, reason codes - fits in one readable file with a 33-test codec suite behind it. Writing a wire protocol from the spec and watching two processes talk is one of the purest joys in this trade.

3. The transport is a seam, so the tests never touch a broker. One contract, three implementations: an in-process hub (used by every test), an MQTT 3.1.1 adapter for maximum compatibility, and the MQTT 5 adapter. A test can therefore assert *exactly-once dispatch across two agents* in milliseconds, with no broker, no network, no flake. Swapping the fabric is a constructor argument.

4. A rune is a plugin, and it takes about 30 lines. There is no special-casing:

`Runes::Plugin.names(kind: :rune)` returns the verb list, and dropping a class in adds a verb to the DSL - no engine change:

```
class Runes::Plugins::Greet < Runes::Rune
  plugin :greet, description: "Say hello"
  # Input / Output / Config + execute(input) -> output
end
```

`greet(:hi) { "world" }` now works in every workflow, and inherits the harness's furniture (journal, registry, observatory) because it is a plugin, not a special case. This is the extension story of DSL applied to a workflow DSL, and it is *satisfying*.

5. We adopted another project's DSL, and made the docs unfakeable. Runes runs Roast workflows unmodified - matched in both directions, including the absences (no `respond`, no `on_error:`, no `.roast/` config). The discipline that follows is the fun part: `examples/analyze_codebase.rb` *is* the Roast README example, and the compatibility test reads that file, so the docs cannot drift from the artifact. When you use a feature and it turns out your own README lied, you fix both - we did, twice, in one afternoon.

6. Reading a fleet is its own reward. The Rails observatory subscribes to `runes/#` and `$a2a/#` and reconstructs every agent's life: cards, online/offline (including the broker's Last Will), prompt lifecycles, tool calls, A2A handoffs. There is a specific moment of delight in opening a browser and watching a handful of agents pass work to each other on a bus you built.

7. The tests are hermetic, and that's a craft, not a chore. Provider HTTP is faked with a real

`Net::HTTPResponse` over a dup transport; the agent CLI is replaced at a factory seam; the chat backend is injected; the whole workflow suite runs with `command_runner=`, `provider_factory=` and `backend=`.

Consequence: **567 tests, 2 593 assertions, no API key, no network, no broker required** - you can run the entire harness on a plane. *(Round-5 audit found this was false: `test/second_audit_test.rb` dialled the default MQTT port 1883. Fixed - it now spawns its own broker on a free port, and the claim is verified: nothing in the suite reaches the network.)*

8. Ruby is the right language for this, and it's not close. Blocks with `instance_exec`, `method_missing` delegation (so `ruby! (:x).anything` falls through to the value), `define_singleton_method` binding a verb's `X/X!/X?` triple per plugin, `Struct` value objects, `Mutex` + `Thread` where Python would need a

framework. The workflow DSL that results is small enough to read in one sitting - mostly `instance_exec`, `define_singleton_method` and good manners.

9. The honest bits are fun too. `parallel` uses threads instead of the `async` gem - so, yes, a **running** iteration cannot be cancelled, and we write that down. `cmd` inherits Roast's shell semantics for a single command string; that's in the docs **and** in a test. Known gaps are first class in `STATE.md`. Nothing builds trust faster than a project that tells you where it's weak.

10. The fleet got a face. The observatory started as a packet list and is now a console: **workflow runs drawn as timelines** (one bar per step, placed by its offset and scaled to the run, because a 24 ms `cmd` next to a 0.2 ms `ruby` should **look** like that), a **fleet topology** with delegation edges whose width is task volume and colour is failure rate, an **interaction waterfall** where the bars are the **gaps** - so a 30-second planner call looks like 30 seconds instead of hiding in a list of timestamps - and a dashboard with a traffic sparkline. All of it from telemetry, none of it guessed.

| Why this is more important than it looks

1. Agents are becoming infrastructure, and infrastructure has requirements. A prompt loop is a demo. The moment an agent can write files, run commands, spend money and talk to other agents, it needs what every other piece of infrastructure needs: **identity, least privilege, audit, and observability**. Runes is built with the guard, the journal, the signatures and the observatory as the product - the model call is the easy part. That inversion is the whole bet, and it's the right one.

2. Non-determinism makes auditability a correctness property. You cannot reproduce an LLM run by re-reading the code. So every prompt lifecycle - request, claim, start, progress, response, plan, tool calls, verifier verdict - is appended to a durable, rotating, `flock`-protected journal and published on the bus, and `bin/runes-replay` shows or replays it. **Your agents' memory should live on your disk, in a format you can read, not in a vendor's dashboard.**

3. "It can do anything" is the threat model, not the feature. A default-deny capability guard decides per tool **and** per action (`fs_read`/`fs_write`/`exec`) on the resolved path; tool RPC is off unless explicitly enabled and authenticated with a shared secret compared in constant time; `run_command` rejects path escapes; untrusted tools can run in a real `ruby.wasm` sandbox with a fuel budget; `bin/runes-acl` generates a fail-closed mosquito ACL file so the **broker** enforces it too. Layers, not vibes.

4. On a shared bus, "unsigned" means "unknown", and unknown means "could be anyone". Ed25519 signed envelopes with a per-agent trust store turn `agent_id` from a claim into a fact. Nothing else in the agent ecosystem treats the message bus as an adversarial surface - and a fleet where any process can impersonate any agent is not a fleet, it's a free-for-all.

5. Interop, or you're an island. Two protocols, deliberately chosen: **A2A** so other ecosystems can discover and task your agents, and **MCP** so your tools are consumable by any MCP client (and vice versa). The transport is swappable; the providers are pluggable; the workflow DSL is Roast-compatible. **Adopting Runes doesn't lock you in - and that's the point.** Tools that only work if you never leave are a trap; we'd rather win on merit.

6. Ruby deserves a first-class answer here. The agent boom happened in Python, and the Ruby community has been told, politely, to enjoy its Rails-shaped past. That's nonsense. Ruby is **better** at the two things agent systems actually are: **expressive DSLs** for describing work, and **boring, reliable libraries** for running fleets. Runes is the argument, in code, that Ruby is a serious choice for agent orchestration - not a nostalgic one.

7. Local-first is a political position as much as a technical one. SQLite, mosquito on `127.0.0.1`, keys in a `.env` you control, no SaaS in the request path. You can run the whole thing on a laptop on a plane, and you can run it on your own broker with your own ACLs. For anything that touches a company's source code, that is not a nice-to-have.

| Receipts

Because "trust me" is not marketing, here is what's actually in the repo today:

<code>`lib/`</code>	**14 265 lines** across 60 files
<code>`test/`</code>	**10 646 lines** across 42 files
Suite	**578 runs, 2 632 assertions, 0 failures** — no keys, no provider calls, no broker required
Observatory	**188 runs, 890 assertions** , **4 314 lines** of Rails 8.1 app code — fleet, runs, topology, traces, who published, and what was refused, all fed through the fleet's own <code>`Runes::Transport`</code>
Executables	**7** : <code>`runes`</code> (TUI), <code>`runes-daemon`</code> , <code>`runes-client`</code> , <code>`runes-mcp`</code> , <code>`runes-replay`</code> , <code>`runes-acl`</code> , <code>`runes-workflow`</code>
Workflow engine	**4 491 lines** total (engine + the seven runes + command runner), stdlib only — no <code>`async`</code> , no <code>`ruby_llm`</code>
The seven runes	<code>`agent`</code> 719, <code>`chat`</code> 503, <code>`repeat`</code> 202, <code>`cmd`</code> 199, <code>`map`</code> 183, <code>`ruby`</code> 86, <code>`call`</code> 68
MQTT 5 adapter	**1 149 lines** , hand-rolled, live-verified against mosquitto 2.1.2 — and it <i>*reconnects and re-subscribes*</i>
Security	**462 lines** of command policy, RPC freshness and a shared nonce cache
Dispatcher	**1 565 lines** , down from 1 968 after the fabric/journal/session split
Gem	builds clean — no secrets, no local state, seven binstubs, MIT <code>`LICENSE`</code> shipped
Live proof	200 messages, one MQTT 5 shared group, **exactly-once distribution** ; plus a dropped-socket proof that the adapter heals and keeps its subscriptions

Run it yourself: `bundle exec rake test`, then `bundle exec ruby demo/smoke.rb` (offline, no key), then `bin/runes-workflow execute examples/analyze_codebase.rb`.

What we're honest about

A pitch that hides the seams is a pitch you'll resent in a month. So:

- **Distribution is exactly-once; execution is at-least-once.** MQTT 5 shared subscriptions put a prompt in front of exactly one group member, but QoS 1 is at-least-once and a reconnect can redeliver work in flight. Handlers are not idempotent yet: a duplicate prompt or task can repeat its side effects. It is written down as the next security-correctness task (a bounded request ledger), not glossed over.
- **Workflow runes ask permission only when asked to.** `RUNES_WORKFLOW_POLICY` puts `cmd`, `agent` and `ruby` behind the capability guard, refusing **before** anything spawns and failing closed on an unreadable policy. It is off by default, because default-deny would break every unmodified Roast file, and its patterns match the command text exactly - so it wants a narrow allowlist, not a wildcard.
- **Token scanning is not a sandbox.** `run_command` refuses interpreters and paths that leave the workspace, but an explicitly allowlisted binary can still escape. Real confinement needs `sandbox-exec/bwrap`.
- **A2A peer cards are unauthenticated** (discovery-only): the agent id is validated, but a retained card from any publisher is accepted.
- **parallel is threads, not async.** So there is no cooperative cancellation of a running iteration. We traded that for one fewer dependency.
- **MQTT 3.1.1 is compatibility-only** - no shared groups, no properties. It refuses loudly, and it now reconnects like MQTT 5 does.
-

on localhost; a prerequisite for the "launch workflows from the browser" idea.

- **One-shot tool feedback:** a single build plan does not yet loop plan->execute->feed-back. Missions compensate with verify-and-resume.
- Every one of those is a `git grep` away from a doc and a test, which is the point.

Write a verb in 30 lines

The extension story, verbatim from `docs/WORKFLOWS.md` and exercised by `test/plugin_test.rb`:

```
class Runes::Plugins::Greet < Runes::Rune
  plugin :greet, description: "Say hello"

  class Input < Runes::Cog::Input
    attr_accessor :name
    def validate! = raise(InvalidInputError, "'name' is required") if name.nil?
    def coerce(value) = (super; @name = value.to_s)
    end

    class Output < Runes::Cog::Output
      attr_reader :text
      def initialize(text) = (super(); @text = text)
      def raw_text = text
      end

      def execute(input) = Output.new("hello #{input.name}")
      end
    end
  end
end
```

No engine change. `greet(:hi) { "world" }` is now a workflow verb, it shows up in `Runes::Plugin.names(kind: :rune)`, and it will appear in the observatory's plugin catalogue as soon as that lands.

Where this goes next

The console's first half shipped (runs, topology, traces, dashboard). The rest of the plan is to finish making the observatory the fleet's **memory, participant and oracle**:

- **Workflow runs, diffed and replayed** - two runs of the same file side by side, and "re-run just the step that failed".
- **Identity badges and impersonation alerts** - signed envelopes already exist; the observatory should prove who published what and shout when one `agent_id` shows up with two keys.
- **An observatory that is an agent** - its own A2A card and an MCP server, so any agent can ask "what happened on the bus?" mid-task.
- **Guard-decision telemetry** - see what the guard *refused*, not just what was published.
- **A broker-free mode** - ingest through the same `Runes::Transport` seam the fleet uses, which also makes MQTT 5 properties visible to the UI.

The full proposal, with costs and risks, is in `docs/OBSERVATORY_ROADMAP.md`.

The paragraph to steal

Runes is a Ruby agent harness that treats the unglamorous half of agent infrastructure as the product: a swappable message fabric (in-process, MQTT 3.1.1, MQTT 5 shared subscriptions), A2A discovery, MCP tools, Ed25519-signed envelopes, a default-deny capability guard, a durable journal with verify-and-resume, and a Rails observatory that watches the whole fleet. Its workflow DSL is Roast-compatible - a Roast file runs unmodified - and every one of its seven verbs is a plugin you can replace in about thirty lines. Twelve thousand lines of library, eight thousand lines of hermetic tests, no network required, no vendor in the path.